

Clustering. Partition-Based Clustering

Modelització Estadística

Grau en Intel·ligència Artificial

Course 2024-25



Outline

- 1 Introduction to Partition-Based Clustering
- 2 K-Means Algorithm
- 3 Practical Implementation of K-Means in R
- 4 Variations and Advanced Topics
- 5 Evaluation metrics
- 6 References

What is Partition-Based Clustering?

Definition

Partition-based clustering divides a dataset into k non-overlapping clusters, where each data point belongs to exactly one cluster.

- Clusters are created to **optimize a predefined objective** function.
- Often **minimizes intra-cluster variance** (*within-cluster inertia*) while maximizing inter-cluster separation (*between-cluster inertia*).

Characteristics

Key Characteristics

- **Flat Clustering**: All clusters are on the same hierarchical level.
- **Predefined k** : The number of clusters k must be set in advance.
- **Iterative Refinement**: Algorithms refine cluster assignments iteratively to optimize the objective.
- **Compact Clusters**: Suitable for data with convex or compact cluster structures.

Limitations

- Sensitive to **initialization of cluster centers**.
- Assumes clusters are roughly **spherical** and **equal-sized**.
- Struggles with **non-convex clusters** or **outliers**.

Applications

Applications of Partition-Based Clustering

- **Marketing:** Customer segmentation for personalized campaigns.
- **Image Processing:** Compression by grouping similar pixel intensities.
- **Document Classification:** Grouping articles with similar topics.
- **Genetics:** Identifying patterns in genetic expressions.

Example

In **customer segmentation**, partition-based clustering can group customers based on attributes like purchase history, demographics, and browsing behavior, enabling targeted marketing strategies.

Overview of K-Means Clustering

What is K-Means?

K-means is a partition-based clustering algorithm that:

- Divides the data into K clusters by minimizing intra-cluster variance.
- Represents each cluster by its **centroid**, the mean of the points in the cluster.
- Iteratively assigns points to the nearest centroid and **updates centroids** until convergence.

Use Cases for K-Means

K-means is widely used in:

- **Anomaly Detection**: Finding outliers as points far from centroids.
- **Pattern Recognition**: Grouping objects with similar features.

K-Means

Goal

Minimize the sum of squared distances between points (x_i) of each i cluster and its centroid (μ_i):

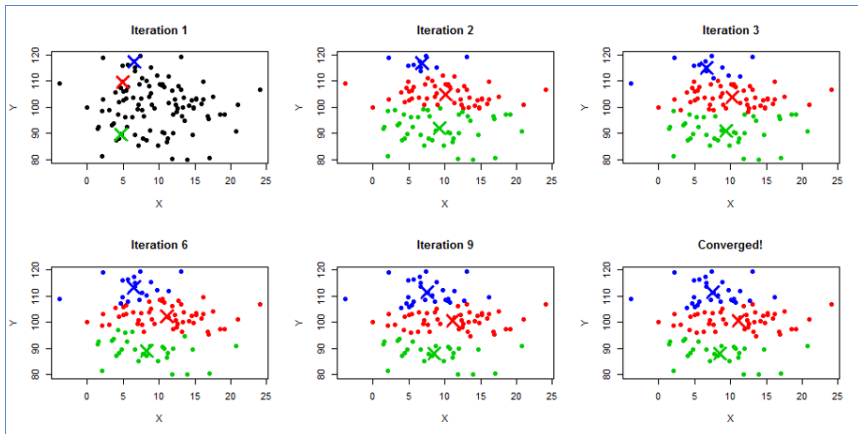
$$J = \sum_{i=1}^K \sum_{x \in C_i} \|x - \mu_i\|^2$$

Steps of the K-Means Algorithm

K-Means clustering follows these iterative steps:

- ① **Initialize Centroids:** Select K initial centroids randomly.
- ② **Cluster Assignment:** Assign each data point to the nearest centroid based on distance (e.g., Euclidean).
- ③ **Centroid Update:** Recalculate the centroid of each cluster as the mean of the points assigned to it.
- ④ **Repeat:** Continue steps 2 and 3 until convergence:
 - Centroids do not change significantly.
 - A maximum number of iterations is reached.

K-means graphically



Strengths & Weaknesses

Strengths of K-Means

- **Simplicity**: Easy to understand and implement.
- **Scalability**: Handles large datasets efficiently.
- **Speed**: Converges quickly for small to medium-sized datasets.
- **Versatility**: Works well for convex, isotropic clusters in numerical data.

Weaknesses of K-Means

- **Initialization Sensitivity**: Poor initialization can lead to suboptimal clustering.
- **Fixed k** : Requires the number of clusters k to be predefined.
- **Assumes Equal-Sized Clusters**: Struggles with clusters of varying sizes or densities.
- **Sensitive to Outliers**: Outliers can distort centroids and cluster assignments.

K-Means Example

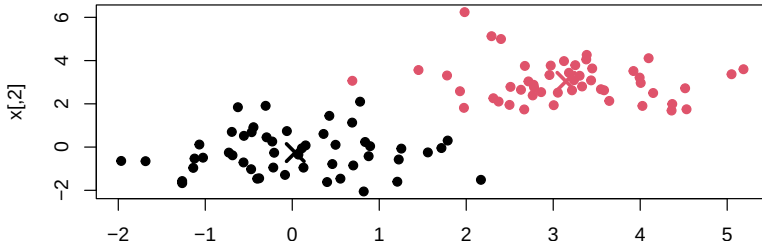
Example

Illustrating K-Means on a 2D Dataset (see next slide):

- **Initialization:** Two random centroids are chosen.
- **Iteration:** Points are assigned and centroids updated until stable clusters are formed.

Example

```
# Toy dataset
set.seed(123); x <- matrix(rnorm(200), ncol = 2)
x[51:100, ] <- x[51:100, ] + 3
# Perform K-means clustering
kmeans_result <- kmeans(x, centers = 2, nstart = 25)
# Plot clusters
plot(x, col = kmeans_result$cluster, pch = 20, cex = 1.5)
points(kmeans_result$centers, col = 1:2, pch = 4, cex = 2, lwd = 3)
```



Convergence

Convergence of K-Means

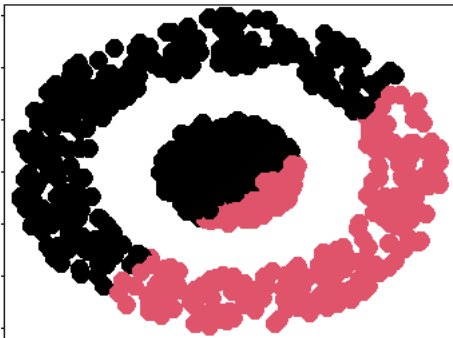
- K-Means converges when centroids **stop changing** or a maximum number of iterations is reached.
- The convergence towards a solution is **guaranteed** since the within-group inertia decreases at each iteration.
- **Convergence to Local Minima**: K-means may get stuck in a local minimum of the objective function.
- The convergence is **fast** (usually less than 5 iterations, even for large amounts of data)
- In general, the algorithm is executed with **different initial partitions** and the most satisfactory solution is retained.
- Due to the random component, it is convenient to set a **seed** before running the algorithm.

Non-Convex data

Non-Convex Issues in K-Means

K-means algorithm struggles to cluster non-convex or irregularly shaped data.

Non-convex data



Practical Implementation of K-Means in R

Step 1: Perform K-Means Clustering with `kmeans()`

The `kmeans()` function in R is the core method for applying K-Means clustering:

- **Syntax:**

```
kmeans(x, centers, nstart, iter.max)
```

- **Key Arguments:**

- **x**: The dataset to cluster (rows = observations, columns = features).
- **centers**: Number of clusters (k).
- **nstart**: Number of random starts to avoid poor initialization.
- **iter.max**: Maximum number of iterations.

- **Output:**

- **Cluster assignments** [`cluster`]: Which cluster each point belongs to.
- **Centroids** [`center`]: Final positions of the cluster centers.
- **Total within-cluster** [`tot.withinss`] sum of squares.

Options of `kmeans` function in R

Versions

- **Hartigan-Wong(1979)**. Default method. Optimizes the distance calculation by dividing the points belonging to updated and non-updated clusters at each iteration.
- **Lloyd B. (1957)**. Explained in previous slides. Simpler method.
- **Forgy(1965)**. Same as Lloyd's but for initial random centroids, scattered points are chosen (more efficient than the previous algorithm)
- **MacQueen (1967)**. Update the centers at every point movement. Use probabilities and asymptotic convergences.

Some K-means packages in R

R packages

- [clusterMixType](#). Categorical and numeric variables.
- [kml](#). Longitudinal data
- [trimcluster](#). Robust methods in the presence of outliers
- [Biganalytics](#). Contains the function `bigkmeansfor` for large data sets

Example (I)

Example

Applying `kmeans()` to the iris dataset:

- Use `nstart` to improve robustness.
- Compare clustering results to known labels for evaluation.

```
# Load and preprocess the data
data <- iris[, -5] # Remove the species column
data_scaled <- scale(data) # Scale features
# Perform K-means clustering
set.seed(123)
kmeans_result <- kmeans(data_scaled, centers = 3, nstart = 25)
# View cluster assignments
kmeans_result$cluster
```

```
##      [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
##      [38] 1 1 1 1 1 1 1 1 1 1 1 1 1 3 3 3 2 2 2 3 2 2 2 2 2 2 2 2 3 2 2
##      [75] 2 3 3 3 2 2 2 2 2 2 2 3 3 2 2 2 2 2 2 2 2 2 2 2 3 2 3 3 3
##     [112] 3 3 2 2 3 3 3 3 2 3 2 3 2 3 3 2 3 3 3 3 2 2 3 3 3 2 3 3 3
##     [149] 3 2
```

Example (II)

```
# Compare clusters with true species  
table(kmeans_result$cluster, iris$Species)
```

```
##  
##      setosa versicolor virginica  
##  1      50           0           0  
##  2       0          39          14  
##  3       0          11          36
```

Visualization

Step 2: Visualize Clusters with factoextra

The factoextra package offers functions to visualize K-Means results:

- `fviz_cluster()`: Plots clusters in reduced dimensions (e.g., PCA).
- `fviz_nbclust()`: Helps determine the optimal number of clusters.

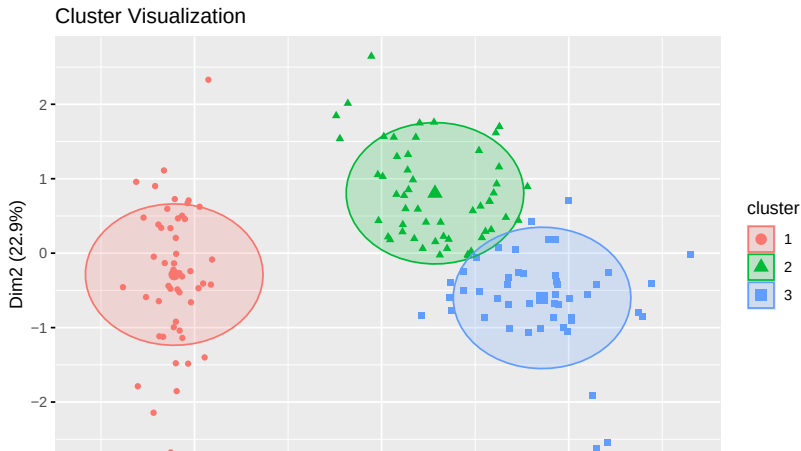
Example

Visualizing clusters with `fviz_cluster()`:

- The plot shows clusters and their centroids in a 2D space.
- Ellipses represent cluster spread.

Visualization

```
library(factoextra)
# Visualize K-means clusters
fviz_cluster(kmeans_result, data = data_scaled,
              geom = "point", ellipse.type = "euclid",
              main = "Cluster Visualization")
```



Number of clusters

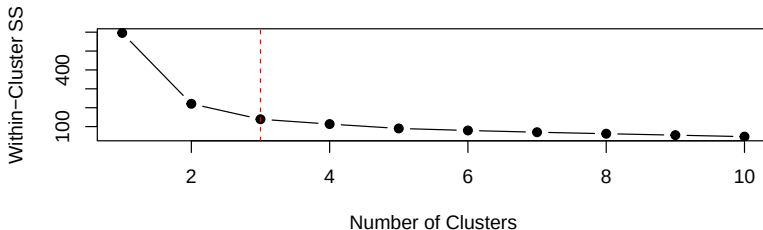
Step 3: Evaluate the Number of Clusters

Use the elbow method or silhouette analysis to determine the optimal k :

- **Elbow Method**: Looks for the "elbow" in the total within-cluster sum of squares plot.
- **Silhouette Analysis**: Measures how well each point fits within its cluster.

Number of clusters

```
# Compute total within-cluster sum of squares for different k
wss <- sapply(1:10, function(k) {
  kmeans(data_scaled, centers = k, nstart = 25)$tot.withinss})
# Plot elbow method
plot(1:10, wss, type = "b", pch = 19, xlab = "Number of Clusters",
     ylab = "Within-Cluster SS"); abline(v=3, lty=2, col="red")
```



Example

The **elbow** at $k = 3$ suggests the optimal number of clusters.

Other functions

Other Useful Functions in R

- **clusplot()** (cluster package): Visualizes clusters in reduced dimensions.
- **silhouette()** (cluster package): Calculates silhouette widths to assess cluster quality.
- **pam()** (cluster package): Performs K-Medoids clustering for robust clustering.

Variants of K-Means

Why Variants of K-Means?

Standard K-Means has limitations such as sensitivity to outliers and poor handling of non-convex clusters. Variants address these issues by:

- Modifying the objective function.
- Improving initialization.
- Increasing robustness to noise and outliers.

Alternative Clustering Algorithms

Alternative Clustering Algorithms

Common alternatives include:

- **K-Medians**: Uses **medians** as centers for robustness against outliers.
- **K-Medoids**: Selects representative objects (**medoids**) to minimize pairwise
 - **PAM**
 - **CLARA**
- **K-Means++**: Improve initialization by choosing centroids that are far apart

K-Medians

K-Medians Clustering

- **Objective:** Minimize the sum of absolute deviations:

$$J = \sum_{i=1}^K \sum_{x \in C_i} |x - m_i|$$

where m_i is the median of cluster C_i .

- **Advantages:**
 - Robust against outliers compared to K-means.
 - Suitable for ordinal or skewed numerical data.
- **Limitations:**
 - Slower convergence compared to K-means.
 - Not suitable for high-dimensional data.

K-Medoids

K-Medoids Clustering

- **Objective:** Minimize the sum of pairwise dissimilarities between objects and their medoid:

$$J = \sum_{i=1}^K \sum_{x \in C_i} d(x, m_i)$$

where m_i is the medoid of cluster C_i .

- **Advantages:**
 - Handles arbitrary distance metrics (e.g., Manhattan, custom).
 - Effective for datasets with noise or outliers.
- **Limitations:**
 - Computationally expensive for large datasets.
 - Requires more iterations for convergence.

K-medoids: PAM

PAM: Partitioning Around Medoids

- **Objective:** Find k representative objects (medoids) that minimize the total dissimilarity between objects and their nearest medoid:

$$J = \sum_{i=1}^k \sum_{x \in C_i} d(x, m_i)$$

where m_i is the medoid of cluster C_i .

- **Process:**
 - 1 Select k initial medoids randomly.
 - 2 Assign each point to the nearest medoid.
 - 3 Iteratively replace medoids with non-medoids to reduce the objective function.
- **Advantages:**
 - Robust to noise and outliers.
 - Handles arbitrary distance metrics.
- **Limitations:**
 - Computationally expensive for large datasets ($O(n^2)$).

K-medoids: CLARA

CLARA: Clustering Large Applications

- **Purpose:** Adapt PAM for large datasets by sampling.
- **Process:**
 - 1 Draw multiple samples (subsets) of the dataset.
 - 2 Apply PAM to each sample to identify medoids.
 - 3 Evaluate clustering quality on the full dataset using medoids from each sample.
 - 4 Choose the best medoid set based on the lowest total dissimilarity.
- **Advantages:**
 - Handles large datasets efficiently.
 - Retains robustness to outliers.
- **Limitations:**
 - Quality depends on the sampling strategy.
 - May miss the global optimum.

K-Means++

K-Means++ Initialization

- **Purpose:** Improve initialization by choosing centroids that are far apart.
- **Process:**
 - 1 Choose the first centroid randomly from the data.
 - 2 Select each subsequent centroid with probability proportional to its squared distance from the nearest existing centroid.
 - 3 Run K-Means with the initialized centroids.
- **Advantages:**
 - Reduces the likelihood of poor local minima.
 - Faster convergence compared to random initialization.
- **Limitations:**
 - Slightly increases the computational cost of initialization.

Methods Comparison

Comparison of Methods

- **K-Means:**

- Fast and effective for convex clusters.
- Sensitive to outliers.

- **K-Medians**

- More robust to outliers.
- Slower convergence.

- **K-Medoids: PAM**

- Accurate but computationally expensive for large datasets.
- Robust to noise and work well with arbitrary dissimilarity metrics

- **K-Medoids: CLARA**

- Scalable and efficient for large datasets but approximate.
- Robust to noise and work well with arbitrary dissimilarity metrics.

- **K-Means++:**

- Improves initialization but increases computational cost.

Other variants

Variants

- **Fuzzy C-means**: Each point has a diffuse degree of belonging to each group.
- **Expectation-maximization**: Gaussian mixture models. It uses a probabilistic allocation to each group, instead of deterministic assignments.
- **KD-trees**: Filtering for improving efficiency at every step of the algorithm.
- **Spherical k-means**: For directional data (angles instead of distances).
- **Minkowski metric weighted k-means**: It solves the problem of noise by assigning weights to the components of the vectors by groups.

Evaluation Metrics for Clustering

Why Evaluate Clustering?

Clustering evaluation helps determine:

- **Internal Validation:** How well-defined the clusters are based on data compactness and separation.
- **External Validation:** How well clustering matches ground truth labels when available.

Internal validation properties

There are different **desirable properties** for assessing the quality of the clustering without any external reference:

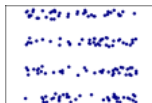
- **Compactness.** Low within-variability



- **Separation.** High between-variability



- **Connectivity.** Up to what point the connected elements belong to the same cluster.



Internal Validation measures

Internal Validation Metrics

- **Silhouette Score:**

$$S(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

- $a(i)$: Average distance to other points in the same cluster.
- $b(i)$: Average distance to the nearest cluster.
- $S(i) \in [-1, 1]$:
 - $S(i) \approx 1$: Point is well-matched to its cluster.
 - $S(i) \approx -1$: Point is misclassified.

- **Dunn Index:**

$$D = \frac{\min_{i \neq j} \text{dist}(C_i, C_j)}{\max_k \text{diam}(C_k)}$$

- Measures separation between clusters relative to cluster size.
- Higher values indicate better clustering.

- **Within-Cluster Inertia (WCSS):**

$$\text{WCSS} = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$

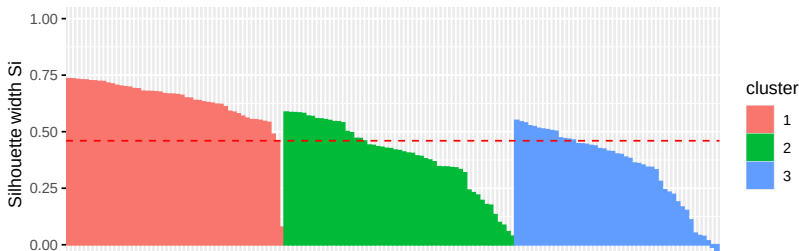
- Measures compactness of clusters.
- Lower values indicate tighter clusters.

Example: Silhouette Index

```
library(cluster)      # For silhouette scores
library(factoextra)   # For clustering visualization
# Calculate silhouette scores
silhouette_scores <- silhouette(kmeans_result$cluster, dist(data))
fviz_silhouette(silhouette_scores) # Visualize silhouette scores
```

```
##   cluster size ave.sil.width
## 1         1   50          0.64
## 2         2   53          0.39
## 3         3   47          0.35
```

Clusters silhouette plot
Average silhouette width: 0.46



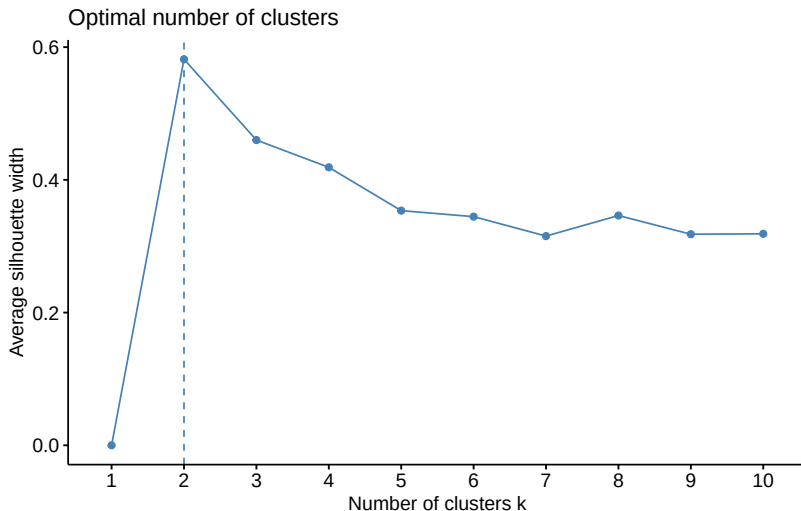
Silhouette index interpretation

Interpretation

- $0.7 < S < 1 \rightarrow$ **strong** cluster tendency
- $0.5 < S < 0.7 \rightarrow$ **reasonable** cluster tendency
- $0.25 < S < 0.5 \rightarrow$ **weak** cluster tendency

Number of clusters with silhouette index

```
fviz_nbclust(data_scaled, FUNcluster=kmeans)
```



External Validation measures

External Validation Metric: Rand Index

- **Rand Index (RI):**

$$RI = \frac{TP + TN}{TP + FP + FN + TN}$$

- Compares cluster assignments to ground truth labels.
- Counts the proportion of correctly grouped or separated pairs.
- $RI \in [0, 1]$: Higher values indicate better agreement.

- **Adjusted Rand Index (ARI):**

$$ARI = \frac{RI - \mathbb{E}[RI]}{\max(RI) - \mathbb{E}[RI]}$$

- Corrects for chance grouping.
- $ARI = 0$: Random clustering, $ARI = 1$: Perfect clustering.

Example: Rand Index

```
# Compute Adjusted Rand Index
library(mclust)
true_labels <- as.numeric(iris$Species)
adjusted_rand <- adjustedRandIndex(kmeans_result$cluster, true_labels)
print(paste("Adjusted Rand Index:", round(adjusted_rand, 2)))
```

```
## [1] "Adjusted Rand Index: 0.62"
```

Tips for evaluation

Practical Tips for Evaluation

- **Silhouette Score**: Ideal for datasets where separation between clusters is critical.
- **Dunn Index**: Use for evaluating separation in datasets with well-defined clusters.
- **Rand Index**: Compare clustering performance when ground truth labels are available.
- **Combine metrics** to get a holistic view of clustering quality.

Clustering in the factorial space: PCA + K-Means

Why we perform the clustering in the factorial space?

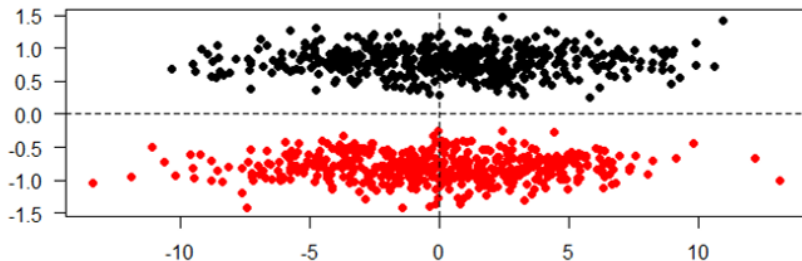
- To take into account the structural component of data and discard the noise
- To reduce the curse of dimensionality
- To work with orthogonal factors (avoiding multicollinearity)
- To embed the points in an Euclidean space

Which disadvantages do we have?

- **Interpretation is more difficult** since we work with factors. Need to come back to the original variables
- Need to **specify the significant directions** of the factorial space. Working with all directions is equivalent to perform the clustering with the raw data

Clustering in the factorial space: Be cautious

- In some cases, PCA can be counterproductive
- In this example, the real clusters (red and black) could be mixed when applying a PCA on one dimension (horizontal axis)



Hierarchical + k-means

- **Option 1:** Useful with big datasets
 - **Stage 1.** A k-means is performed with a large number of groups (e.g., 100)
 - **Stage 2.** A hierarchical clustering with the gravity centers of previous groups is performed.
- **Option 2:** Useful with few observations. In certain cases, it improves performance
 - **Stage 1.** Hierarchical clustering
 - **Stage 2.** Starting from an initial partition of this hierarchical clustering, points can be reallocated according to the algorithm of k-means

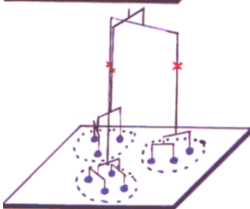
Hierarchical + k-means (Option 1)



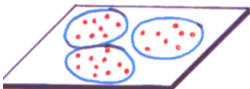
Data to classify



Stage 1
Partition using K-means with
a higher K



Stage 2
Hierarchical classification with
the centroids



Classified data

References

- A. Kassambara, Practical guide to cluster analysis in R: Unsupervised machine learning. Sthda, 2017, vol. 1.
- Hastie, T., Tibshirani, R. & Friedman, J. H. The Elements of Statistical Learning Data Mining, Inference, and Prediction. Springer (2009)

Appendix: Gower Distance

What is Gower Distance?

Gower distance is a similarity measure that works with **mixed data types**, including:

- **Numerical Variables**: Treated using normalized absolute differences.
- **Categorical Variables**: Based on matching or mismatching of categories.
- **Ordinal Variables**: Scaled to preserve order relationships.

It computes a normalized distance value for each variable and combines them to produce an overall similarity score.

Appendix: Gower Distance. Formula

Formula for Gower Distance

For two objects x and y , the Gower distance is calculated as:

$$d(x, y) = \frac{\sum_{i=1}^p w_i d_i(x, y)}{\sum_{i=1}^p w_i}$$

where:

- $d_i(x, y)$: Scaled distance for variable i .
- w_i : Weight for variable i (1 if valid, 0 if missing).
- p : Total number of variables.

Appendix: Gower Distance. Formula

Formula for Gower Distance

Categorical data for variable i and individuals x, y :

- $d_i(x, y) = 0$ if $x_i = y_i$
- $d_i(x, y) = 1$ if $x_i \neq y_i$

Continuous data for variable i and individuals x, y :

$$d_i(x, y) = 1 - \frac{|x_i - y_i|}{R_i}$$

where R_i being the range of variable i .

Appendix: Gower Distance. Why Use this Distance?

Why Use Gower Distance?

- **Versatility:** Handles datasets with numerical, categorical, and ordinal variables.
- **Robustness:** Accommodates missing values using weights (w_i).
- **Scalability:** Useful for clustering and similarity analysis in heterogeneous data. There is no need to scale the data.